

QA Report

Tick & Taka — personal tasks + money app (Android / Bun API)

Project	Tick & Taka (monorepo: apps/api, apps/mobile, packages/shared)
Date	6 October 2026
Commit	<code>3b5438b</code> (main) — “feat(api): rewrite public pages for Google OAuth verification”
Auditor role	QA Lead
Mode	Read-only. No source, config, dependency or git state changed.
Companion reports	UX_Report.pdf, Code_Quality_Report.pdf (written separately)

Severity scale used in all three reports: **Critical** data loss, security hole, crash or broken core flow · **High** major feature broken or a stated principle clearly violated · **Medium** real problem with a workaround, notable friction or debt · **Low** polish, consistency, minor cleanup.

1. Executive summary

58 / 100 overall QA score

The static gates are clean: Biome reports no diagnostics, TypeScript passes in all three workspaces, and all 185 tests pass (API 100, shared 71, mobile 14). Money is stored as integer minor units, each user has a separate SQLite file, every API route except `/health`, the public pages and `/auth/google` requires a token, and the safe-to-spend formula matches the spec's own worked example. Those choices are sound.

The weak spot is the offline write path, which the product promises and which the tests never touch. Queued writes are dropped after three quick retries when the phone has a network but cannot reach the server, and they are dropped at once on any 5xx. An edit made offline to a record that was also created offline is silently thrown away when the queue replays. An expired 30-day session wipes whatever is still queued. On the capture side, Bangla digits are not parsed, so “`ଟ ୨୦`” is saved as a task and never reaches the AI fallback. Six further High issues (bill pay replay, budget rollover, offline timer, app-lock bypass and two of the above) all go against the spec's “Honest numbers” principle.

Severity counts

Severity	Count
Critical	2
High	6
Medium	9
Low	9
Total	26

Top 5 to fix first

1. **QA-001** **Critical** The offline outbox gives up after 3 retries (or at once on 5xx), so writes are lost when the server is unreachable, restarting or behind a captive portal.
2. **QA-002** **Critical** Create a record offline, then edit it offline: on replay the server stamps the create with “now” and last-write-wins discards the edit. Reproduced.
3. **QA-011** **High** Quick-add ignores Bangla digits. “`ଟ ୨୦`” and “`lunch ୨୫୦`” become tasks with high confidence, so the AI fallback is never called. Reproduced.
4. **QA-007** **High** Replaying “Paid” with the same `transactionId` advances the bill's due date a second time, so a month is silently skipped. Reproduced.
5. **QA-017** **High** On a cold start the fingerprint lock reads its setting before it has loaded, so the app opens unlocked.

2. Scope & method

What was reviewed

- **Intent documents:** `PRODUCT.md` , `DESIGN.md` , `README.md` , `docs/IMPLEMENTATION_PLAN.md` and the 37-page spec (`tick_taka_spec.pdf` , read via `_raw/spec.txt`). Requirements were turned into the traceability table in section 3.
- **Shared logic:** `packages/shared/src` (money, dates, finance, recurrence, quick-add, calculator, gamification, schemas), read in full.
- **API:** `app.ts` , `middleware`, `lib/crud.ts` , `lib/money-queries.ts` , and the budgets, today, transactions, accounts, recurring, tasks, time-entries, goals, debts, shopping, sync, auth, uploads and AI modules (usage, client, routes, parse, assistant loop), jobs, the env schema and the Nginx config.
- **Mobile:** `src/lib` (query client and outbox, http, api, auth, user-data, server-clock, optimistic-spend), root layout, quick-add, Today screen, task actions, bill pay buttons, timer/focus, app lock, notifications, receipts.
- **Tests:** all 21 test files were listed, and the money, quick-add and helper tests were read to judge assertion quality.

Commands and evidence

Command	Result	Source
<code>bunx biome check .</code>	388 files, exit 0, no diagnostics	<code>_raw/lint.txt</code> (discovery phase)
<code>bun run typecheck</code>	shared / api / mobile exit 0	<code>_raw/typecheck.txt</code>
<code>bun run test</code>	185 pass, 0 fail (api 100, shared 71, mobile 14)	<code>_raw/test.txt</code>
coverage per workspace	api 93.6% lines, shared 88.8%, mobile 71.6% (pure modules only)	<code>_raw/coverage-*.txt</code>
<code>bun audit</code>	6 advisories (2 high, 4 moderate), all transitive via Expo/Tailwind/drizzle-kit	<code>_raw/audit.txt</code>
QA reproduction scripts (Bun, in-memory SQLite via the project's own <code>createTestContext</code>)	8 behaviours reproduced; output in Appendix B	temporary <code>_qa_tmp/</code> (deleted)

What could not be verified

- Nothing was run on a device or emulator, so on-device behaviour (widget, shortcuts, notifications firing, biometric prompt, the background/foreground lifecycle) was judged from code alone. Findings that depend on it carry the `Unverified` tag or say so.
- The Nginx / pm2 deployment was not exercised; proxy timeouts are judged from `deploy/nginx-tick-taka.conf` and Nginx defaults.
- No real OpenAI calls were made. AI behaviour was judged from the code and the project's fake AI client.
- Whether the `bun audit` advisories are reachable at runtime was not assessed (see the Code Quality report).

3. Requirements traceability

Requirements come from the spec's build checklist (pp. 34–37), its technical rules (pp. 17–26) and PRODUCT.md. Status: **Implemented** **Partial** **Missing** **Broken**. “Implemented” means the code path exists and is covered by API tests or was read end to end. It does not mean it was exercised on a device.

Requirement	Implementation	Status	Notes / finding
Sign-in, JWT, per-user SQLite + migrations	api/modules/auth/routes.ts, middleware/auth.ts, db/user-registry.ts	Implemented	Google sign-in replaces the spec's password. Session is 30 days with no refresh (QA-003).
Auth on every protected route	app.ts:38-66 (requireAuth on the whole api sub-app)	Implemented	Reproduced: an unauthenticated /budgets/safe-to-spend returns 401. Public routes: /health, /, /privacy, /terms, /auth/google.
Offline cache and queued writes, in order	mobile/lib/query-client.ts, outbox.ts, _layout.tsx	Broken	Writes are lost on unreachable server, 5xx or 401, and offline edits get dropped (QA-001, -002, -003, -004).
Conflicts: last write wins on updated_at	api/lib/crud.ts:82-92, mobile/lib/server-clock.ts	Partial	The rule compares a phone edit time with a server create time (QA-002).
GET /sync/changes? since=	api/modules/sync/routes.ts: 11-31	Partial	The endpoint exists but the app never calls it (QA-006).
Quick-add on-phone parser (incl. Bangla)	shared/quick-add.ts, calculator.ts	Partial	English works. Bangla digits fail, and words like “weekly” create false repeats (QA-011, -012).
Tasks, inbox, top three, areas/projects, subtasks (1 level)	api/modules/tasks/service.ts	Implemented	Undo of delete loses subtasks (QA-016).
Repeat on completion	tasks/service.ts:71-138, 228-250	Partial	Undo of completion leaves a duplicate (QA-015).
Recurrence in plain words	shared/recurrence.ts	Partial	Rules that never occur still pass validation (QA-014).
Overdue rescue, does-my-day-fit, evening, someday, logbook, Eisenhower, energy	tasks/service.ts:265-369; mobile app/plan/*	Implemented	Not exercised on a device.
Focus timer / time tracking	time-entries/service.ts, features/focus, features/time	Partial	An offline start/stop records the wrong hours (QA-004).
Habits, streaks, 2 freeze days a month	shared/gamification.ts, habits/service.ts	Implemented	Unit tested.
Routines	routines/service.ts, features/routines	Implemented	—

Requirement	Implementation	Status	Notes / finding
Accounts, computed balances, balance check	accounts/service.ts, lib/money-queries.ts:11-46	Implemented	Balances are computed from transactions, never stored.
Transactions incl. transfer fees, cross-currency, cursor paging	transactions/service.ts	Implemented	—
Budgets: 3 buckets, rollover	budgets/service.ts:73-179	Partial	Rollover stops after one inherited month, and foreign-currency spending is mixed in (QA-008, -009).
Safe to spend = flexible left ÷ days left incl. today	shared/finance.ts:20-24, budgets/service.ts:195-214	Implemented	Matches the spec example (tested). Stale after midnight (QA-013).
Pace alert	shared/finance.ts:40-58, today/service.ts	Implemented	—
Bills & subscriptions, "Paid" creates transaction	recurring/service.ts:95-146	Partial	Not idempotent on replay (QA-007).
Expected income with FX conversion	recurring/service.ts:33-41, 108-120	Implemented	Tested.
Savings goals, goals become tasks	goals/service.ts	Implemented	—
Debts and payoff forecast	debts/service.ts, shared/finance.ts:88-99	Implemented	—
Calculator keypad ("1850/3 saves ₮617")	components/ui/amount-keypad.tsx, shared/calculator.ts	Partial	Saves ₮616.67 (QA-010).
SMS auto-capture and review panel	— (no module; README:291 says SMS permissions are blocked)	Missing	Deliberately dropped, but the plan and spec still list it (QA-024).
Events, shopping list, money calendar, spotter, payday plan, recap, reviews, area dashboard, hourly rate, cost in hours	api modules events, shopping, insights, reviews; mobile app/money/*, app/review/*	Implemented	Present and wired. Most are API-tested; screens not run.
Home-screen widget, launcher shortcuts	features/widget, features/app/use-app-shortcuts.ts	Implemented Unverified	Needs a device.
Local notifications, quiet hours, bill reminder 2 days before	features/notifications/*	Partial	Gaps in what gets scheduled (QA-022).
Nightly backup (keep 14) and JSON export	jobs/backup.ts, sync/routes.ts (export)	Implemented	No catch-up if the server is down (QA-023).
Fingerprint lock after 5 min in background; privacy mode	features/security/app-lock.tsx, lib/privacy.ts	Broken	Lock is skipped on cold start (QA-017).

Requirement	Implementation	Status	Notes / finding
Login limited to 5 attempts / 15 min / X-Real-IP; API on 127.0.0.1	auth/routes.ts:24-27, env.ts:5	Partial	Limit is 10, and HOST defaults to 0.0.0.0 (QA-018).
AI features (parse, receipt, categorise, plan day, break down, coach, ask, budget suggestions)	api/modules/ai/*	Implemented	Fake-client tested. /ai/ask is reached through the assistant.
AI guardrails: cap in Settings, graceful fallback, never saves without confirmation	ai/usage.ts:59-92, assistant/agent.ts, prompts.ts:46	Partial	Diverges from the spec (QA-019, -020).
Tiki, sparks, levels, streak freezes	shared/tiki.ts, gamification.ts	Implemented	—

4. Findings

Findings are grouped by category. Categories are ordered by their worst finding, and findings within a category by severity. Every line reference below was opened and checked at commit 3b5438b. "Reproduced" means a Bun script that calls the project's own code produced the output shown (full output in Appendix B).

4.1 Offline sync & data integrity

QA-001 · The offline outbox gives up and discards writes when the server is unreachable or returns 5xx

Critical

Category: Offline / data loss Effort: M

Where: `apps/mobile/src/lib/query-client.ts:L55-L58` (retry policy), `:L83-L85` (online = "has a network"), `apps/mobile/src/lib/http.ts:L69`, `apps/mobile/src/lib/api.ts:L64`; `apps/mobile/src/app/_layout.tsx:L88-L91` (persistence).

What's wrong: Every write goes through the `["outbox"]` mutation. Its retry policy is `!(error instanceof ApiError) && count < 3`. TanStack only *pauses* a mutation when `onlineManager` reports offline, and here "online" means `state.isConnected !== false`. When the phone has Wi-Fi or mobile data but the API can't be reached (VPS down, `pm2` restart, captive portal, weak signal), the request fails with `ServerUnreachableError`, is retried 3 times over about 7 s, then errors for good. The entry leaves the queue and only a toast remains. Any 5xx (Nginx 502 during a restart, a 500) is an `ApiError`, so it is not retried at all. A mutation that is running or retrying (not paused) is also not persisted by the default `shouldDehydrateMutation`, so killing the app mid-retry loses it too. The README promises the opposite: "Failed requests retry on their own", "every write goes through one queue that replays in order after a restart".

```
// query-client.ts L55-58
queryClient.setMutationDefaults(["outbox"], {
  mutationFn: (request: OutboxRequest) => send(request.method, request.path,
    request.body),
  scope: { id: "outbox" },
  retry: (count, error) => !(error instanceof ApiError) && count < 3,
// L83-85
onlineManager.setEventListener((setOnline) =>
  NetInfo.addEventListener((state) => setOnline(state.isConnected !== false)),
```

Evidence: code reading plus TanStack Query 5.104.1's `canFetch` (`node_modules/@tanstack/query-core/build/modern/retryer.js:L9-L11`), which only pauses when `onlineManager.isOnline()` is false. **Unverified on device**. Steps: put the phone on Wi-Fi, stop the API (`pm2 stop`), log "cha 20", wait 10 s, start the API. Expected: the expense syncs. Predicted: a "Couldn't save the transaction" toast, and the expense never reaches the server.

Why it matters: Quick capture is the core flow, and the owner logs in short bursts on rickshaws and at shop counters, exactly where reachability is flaky. Lost expenses break "Honest numbers", and the user isn't told what was lost beyond a passing toast.

Recommendation: Treat network errors, timeouts, 408/429 and 5xx as "pause and retry later". Retry forever with capped backoff, and give up only on 4xx validation errors. Persist the queue yourself, or set `dehydrateOptions.shouldDehydrateMutation: () => true` for the outbox key. Keep any permanently failed write in a "Needs attention" list that the user can retry or edit.

```
retry: (count, e) => !(e instanceof ApiError) || e.status >= 500 || e.status === 429,  
retryDelay: (n) => Math.min(60_000, 1000 * 2 ** n),
```

QA-002 · An offline edit to a record created offline is silently discarded on replay

Critical

Category: Sync / last-write-wins Effort: S

Where: `apps/api/src/lib/crud.ts:L71-L92`; every sheet that sends `updatedAt: editTime()`, e.g. `apps/mobile/src/features/tasks/task-sheet.tsx:L179`, `features/money/transaction-sheet.tsx:L152`.

What's wrong: `create()` stamps `updatedAt` with the server's clock *when the queue replays*. The following PATCH carries the phone's edit time from when the user actually made the edit, which is earlier. `update()` returns the row unchanged when `editedAt < current.updatedAt`, with status 200 and no error. Create a task or transaction offline, fix a typo or the amount, reconnect later, and the fix is lost. The optimistic cache shows the edit until the next refetch, then it reverts.

```
// crud.ts L82-85  
update(id: string, values: UpdateValues<T>): Row {  
  const current = service.get(id) as Row & { updatedAt: number };  
  const { updatedAt: editedAt, ...changes } = values;  
  if (editedAt !== undefined && editedAt < current.updatedAt) return current;
```

Evidence (reproduced): create task `Buy milk`, PATCH title with an edit time 1 min after creation, replayed 2 h later → PATCH status: 200, title after replay: "Buy milk".

Why it matters: Corrections to money entries vanish silently, which breaks the reconciliation promise. It happens on the normal path: offline capture, then a fix.

Recommendation: Let the client send its own `createdAt` / `updatedAt` on create (server time via `editTime()`) and store it, or have `create()` set `updatedAt = min(now, client edit time)`. Add a replay test: create, then edit, then replay much later.

QA-003 · An expired session or tapping “Sign out” wipes queued offline writes

High

Category: Offline / auth Effort: M

Where: `apps/api/src/modules/auth/routes.ts:L11` (30-day token, no refresh);
`apps/mobile/src/lib/http.ts:L49`; `apps/mobile/src/lib/auth.ts:L103-L111`;
`apps/mobile/src/lib/user-data.ts:L20-L30`; `features/settings/account-section.tsx:L38`.

What's wrong: The JWT expires after 30 days and nothing refreshes it. The first 401 calls `signOut()`, which runs `clearUserData(): queryClient.getMutationCache().clear()` plus removal of every non-device `AsyncStorage` key. Any writes still queued (for example after a day offline that happens to cross day 30) are deleted before they are sent. The Settings “Sign out” button does the same with no check of `usePendingWrites()`.

```
// auth.ts (mobile) L111  
connectAuth({ token: () => tokenStore.get(), onUnauthorized: () => void signOut() });  
// user-data.ts L21-22  
await queryClient.cancelQueries();  
queryClient.getMutationCache().clear();
```

Why it matters: Silent data loss on a predictable schedule, once a month.

Recommendation: Issue a sliding session: refresh when less than 7 days remain, or return a new token in a header. On 401, pause the outbox and ask the user to sign in again *without* clearing it, and replay after re-authenticating as the same user id. Before a manual sign-out with pending writes, warn the user and offer to sync first.

QA-004 · An offline timer start records the reconnect time, and the matching stop is rejected

High

Category: Offline / time tracking Effort: S

Where: `apps/mobile/src/features/time/time-tracking.tsx:L122-L127`;
`apps/api/src/modules/time-entries/service.ts:L94, :L104-L110`; also `features/timer/running-timer-bar.tsx:L53-L58` (stop uses the phone's `Date.now()`, not `editTime()`).

What's wrong: The time-tracking Start button sends no `startedAt`, so the server uses its own `now`, which is the replay time. The Stop that follows carries the real tap time. That is earlier than the server's start, so the server answers `400 Stop time must be after the start`, the outbox drops the stop, and the timer keeps running on the server. The focus screen does send `startedAt`, so only manual time tracking is affected.

Evidence (reproduced): start offline, stop 45 min later, replay 2 h later → `start 201 startedAt - tapStart (min): 120; stop 400 {"error":{"code":"bad_request","message":"Stop time must be after the start"}}; still running on server: true`.

Why it matters: Tracked hours feed the effective hourly rate and cost-in-hours. The spec says “tracked hours always reconcile”.

Recommendation: Always send `startedAt: editTime()` on start and `endedAt: editTime()` on stop. Make stop target the entry id (`POST /time-entries/:id/stop`) so a replay is idempotent.

QA-005 · Retried action endpoints aren't idempotent and show false error toasts

Low

Category: Sync / idempotency Effort: S

Where: `apps/api/src/modules/shopping/routes.ts:L55-L66` (checkout → 400 “Tick at least one item first” on replay); `time-entries/service.ts:L106` (stop → 409); `lib/crud.ts:L101-L104` (restore → 409 “is not deleted”).

What's wrong: If the request reached the server but the response was lost to a timeout, the outbox (or a double tap) sends it again. These endpoints reject the second call with a 4xx, and `MutationCache.onError` shows “Couldn't ...” even though the first call succeeded.

Why it matters: The user is told something failed when it didn't, may redo it by hand, and ends up with duplicates.

Recommendation: Key these actions on a client id. For example, checkout with an already-used `transactionId` returns the existing transaction, and restore of a live row returns 200.

QA-006 · `/sync/changes` exists but the app never uses it; every write refetches everything

Low

Category: Sync / spec traceability Effort: M

Where: `apps/api/src/modules/sync/routes.ts:L11-L31`; `apps/mobile/src/lib/query-client.ts:L23-L27` (`invalidateQueries()` with no filter, 250 ms after every write). A grep for `sync/changes` or `sync.changes` in `apps/mobile/src` finds nothing.

What's wrong: The spec lists `/sync/changes` for refreshing the offline cache. The app instead invalidates every query after each write, and screens that aren't open keep stale data until they're next opened online.

Why it matters: More data used on mobile, and stale offline screens after edits made through the assistant or on another surface. Low impact for one user.

Recommendation: Either remove the endpoint from the spec and README, or use it on reconnect to patch cached lists.

4.2 Money, budgets & bills

QA-007 · Replaying “Paid” advances a bill's due date twice, skipping a whole period

High Category: Money / idempotency Effort: S

Where: `apps/api/src/modules/recurring/service.ts:L95-L146`; callers `apps/mobile/src/features/today/next-up.tsx:L35-L46`, `features/money/recurring-sheet.tsx:L107-L124`.

What's wrong: `pay()` inserts the transaction with `onConflictDoNothing()`, so it is idempotent on `transactionId`, but it then *always* runs `base.update(id, { nextDueAt: advance(item) })`. A retried request moves the due date forward again. Separately, the Today “Paid” button generates a fresh `newId()` on every tap and doesn't remove the row optimistically, so a double tap while offline logs the bill twice. It also routes only *income* in a foreign currency to the sheet, so a USD bill paid from Today fails with “Enter the USD → BDT rate”.

```
// recurring/service.ts L123-L143 (trimmed)
db.insert(transactions).values({ id: transactionId, ... }).onConflictDoNothing().run();
...
const updated = base.update(id, { nextDueAt: advance(item, timeZone), overdueAt: null });
```

Evidence (reproduced): bill due 5 Oct, pay twice with the same `transactionId` → first pay nextDue “2026-11-05...”, replay nextDue “2026-12-05...”, expense rows: 1.

Why it matters: November's bill vanishes from Today and from notifications, which is the late-fee scenario the feature exists to prevent.

Recommendation: When `transactionId` already exists with `recurringId = id`, return the current state without advancing. Store the paid occurrence date on the transaction and advance only from that date. On Today, hide the row optimistically and disable the button after one tap.

QA-008 · Budget rollover stops after the first month that inherits limits

High Category: Money / budgets Effort: S

Where: `apps/api/src/modules/budgets/service.ts:L73-L91` (inheritance), `:L125-L139` (rollover).

What's wrong: A month with no budgets of its own inherits the latest earlier month's rows. Rollover reads the *previous calendar month's own* rows via `budgetsFor(deps, previousMonth)`, and those are empty when the previous month was itself inherited. Set budgets once in September

with rollover on: October carries in, but November, December and later carry nothing, while the line still shows `rollover: true`. Unspent money silently disappears from flexible-left and therefore from safe-to-spend.

```
// L125-L139 (trimmed)
const previousMonth = addMonths(month, -1);
const previousBudgets = new Map(budgetsFor(deps, previousMonth).map(...)); // ignores inheritance
const carriedMinor = budget?.rollover && previous?.rollover
  ? Math.max(0, previous.limitMinor - (previousSpent.get(cat.id) ?? 0)) : 0;
```

Evidence (reproduced): 2026-10 inherited=true {"limit":300000,"carried":300000}, 2026-11 inherited=true {"carried":0,"rollover":true}, 2026-12 ... {"carried":0}.

Why it matters: The tested path (re-saving budgets each month) works, but the default path of setting budgets once does not. The daily number is understated.

Recommendation: Use `effectiveBudgets(deps, previousMonth).rows` for the previous month. Also decide whether carry should compound (today it ignores the previous month's own carry). Add a 3-month inheritance test.

QA-009 · Foreign-currency spending is counted as taka in budgets and safe-to-spend

Medium

Category: Money / currency Effort: M

Where: `apps/api/src/lib/money-queries.ts:L61-L109` (`spendingRows SUMS amountMinor` across accounts), `modules/budgets/service.ts:L94-L108`.

What's wrong: The spec allows any account to hold another currency, and the test fixtures include a USD "Payoneer" account. `spendingRows` ignores the account currency, so \$50 (5,000 cents) adds 5,000 poisha (₳50) to the flexible bucket. Net worth handles this correctly by excluding foreign accounts (`insights/summary.ts:L120-L141`); budgets don't.

Evidence (reproduced): after a \$50 USD Food expense, `spentTodayMinor: 5000` and `leftToday 42800 → 37800` (a ₳50 drop instead of roughly ₳6,000).

Why it matters: Card or Payoneer spending is under-reported by about 120×, so safe-to-spend overstates what's left.

Recommendation: Either exclude non-default-currency accounts from `spendingRows` and show "+ \$50 not in budget", or store a converted `baseAmountMinor` at entry time, as is already done for foreign income.

QA-010 · The calculator keypad saves ₳616.67 where the spec and the code comment say ₳617

Low

Category: Money / spec mismatch Effort: S

Where: `apps/mobile/src/components/ui/amount-keypad.tsx:L18, :L44-L46` ; `packages/shared/src/money.ts:L28-L33` ; spec p. 13 ("I type 1850/3 ... it saves ₳617"); `packages/shared/test/quick-add.test.ts:L87-L90` asserts 61,667.

What's wrong: `toMinor(616.666...)` rounds to the poisha (61,667). The spec, and the component's own doc comment, promise whole taka.

Why it matters: Cosmetic, but it shows up as "₳616.67" on every split bill.

Recommendation: Pick one. Either round expressions with an operator to whole taka (`Math.round(v) * 100`) or update the spec and comment.

4.3 Capture, parsing & dates

QA-011 · Quick-add doesn't understand Bangla digits; Bangla expenses are saved as tasks and skip the AI fallback

High Category: Capture / localisation Effort: S

Where: `packages/shared/src/calculator.ts:L25, :L114-L116`; `packages/shared/src/quick-add.ts:L172-L181, :L227-L242, :L373-L386`; `packages/shared/src/money.ts:L74-L82`; `apps/mobile/src/features/quick-add/use-quick-add.ts:L87-L96`.

What's wrong: All numeric regexes use ASCII `\d`, and nothing in the repo maps `০-৯` to `0-9` (a search for Bangla digits in source finds only an AI voice hint). “`চা ২০`” isn't seen as an amount, so it falls through to `parseTask`. That function marks confidence “`high`” unless the title contains an ASCII number, and the AI parser is only called for low-confidence drafts, so it is never consulted. The same happens to “`২h thesis`” (not a time entry) and to `parseAmountToMinor("২৫০")`, which returns null.

Evidence (reproduced):

```
parseQuickAdd("cha 20")
{"kind":"expense","amountMinor":2000,"confidence":"high"}
parseQuickAdd("চা ২০")      {"kind":"task","title":"চা ২০","confidence":"high"}
parseQuickAdd("lunch ২৫০")  {"kind":"task","title":"Lunch ২৫০","confidence":"high"}
parseQuickAdd("রিকশা ৬০ bkash") {"kind":"task","title":"রিকশা ৬০
bkash","confidence":"high"}
parseQuickAdd("২h thesis")   {"kind":"task","title":"২h thesis","confidence":"high"}
parseAmountToMinor("২৫০")   null
```

Why it matters: README promises “Bangla and English, typed or spoken, in quick-add”. A Bangla keyboard types Bangla digits by default, so the five-second expense log silently becomes an inbox task.

Recommendation: Normalise digits at the top of `parseQuickAdd`, `evaluateExpression` and `parseAmountToMinor`: `text.replace(/[০-৯]/g, d => String(d.charCodeAt(0) - 0x09E6))`. Also treat any non-ASCII-only task title as “`low`” confidence so Bangla text reaches the AI. Add Bangla cases to `quick-add.test.ts`.

QA-012 · Common words like “weekly” or “monthly” turn a task into a repeating one, and stray numbers become expenses

Medium Category: Capture / parser false positives Effort: S

Where: `packages/shared/src/recurrence.ts:L365-L379` (bare `daily|weekly|monthly|yearly|annually`); `packages/shared/src/quick-add.ts:L307-L311, :L232-L239`.

What's wrong: The final recurrence pattern matches the bare adjective anywhere in the text, strips it from the title and sets an RRULE with high confidence. Separately, a trailing number in task-like text is read as an expense.

```

parseQuickAdd("submit weekly report") {"kind":"task","title":"Submit
report","rrule":"FREQ=WEEKLY","confidence":"high"}
parseQuickAdd("pay monthly rent") {"kind":"task","title":"Pay
rent","rrule":"FREQ=MONTHLY","confidence":"high"}
parseQuickAdd("read chapter 5")
{"kind":"expense","amountMinor":500,"confidence":"low"}

```

Why it matters: One-off tasks start repeating forever and the title loses meaning. The preview line does show “Repeats”, which is a partial workaround. Offline, “read chapter 5” is shown as a 5 expense.

Recommendation: Only accept the bare adjectives at the start or end of the text, or after “every”/“repeat”. Keep the word in the title. Mark recurrence parsed from bare adjectives as “low” so the AI or the user confirms it.

QA-013 · Today doesn't roll over at midnight while open or offline, and new-day expenses don't move the card

Medium

Category: Dates / midnight rollover Effort: S

Where: `apps/mobile/src/lib/queries.ts:L6 , :L32-L37 (key ["today","now"] );`
`apps/mobile/src/lib/query-client.ts:L32-L34 ; apps/mobile/src/lib/optimistic-spend.ts:L41 ;`
`apps/mobile/src/app/(tabs)/index.tsx:L17-L56 .`

What's wrong: Today's cache key doesn't include the date, and data refreshes only on focus or after a write. With the app left open past midnight, or opened offline the next morning, it shows yesterday's top three, timeline and “spent today”. `withNewExpense` skips any expense whose local date $\neq$ `data.date`, so a 00:30 expense doesn't reduce the stale card at all. The header does print `data.date`, but nothing says the data is stale.

Evidence: code reading. Unverified on device.

Why it matters: The shop-counter number is wrong at the start of each day until a refetch succeeds.

Recommendation: Key the query on the local date (`keys.today(toLocalDate(now, tz))`), schedule a refetch at local midnight, and show “Offline · showing yesterday” when `data.date` $\neq$ today.

QA-014 · Repeat rules that never occur pass validation; paying such a bill fails

Low

Category: Dates / validation Effort: S

Where: `packages/shared/src/schemas/common.ts:L20 ; packages/shared/src/recurrence.ts:L63-L108 , :L110-L113 , :L298-L310 ; apps/api/src/modules/recurring/service.ts:L57-L64 .`

What's wrong: `isValidRRule` only checks FREQ. “every month on the 0th” parses to `BYMONTHDAY=0`, and `resolveMonthDay(0)` returns `dim + 1`, which never matches. `nextOccurrence` scans 4,392 days and returns null, so “Paid” returns 400 “This repeat rule has no next date” and a repeating task stops repeating without a word.

Evidence (reproduced): `parseRecurrence("every month on the 0th")` →
`"FREQ=MONTHLY;BYMONTHDAY=0" ; nextOccurrence(...) → null .`

Recommendation: Validate ranges (BYMONTHDAY 1–31 or -1, BYMONTH 1–12, BYHOUR 0–23) in `parseRRule`, and require `nextOccurrence(rule, today) !== null` in the schema refinement.

4.4 Tasks

QA-015 · Undoing completion of a repeating task leaves a duplicate and breaks the series

Medium

Category: Tasks / undo Effort: S

Where: `apps/api/src/modules/tasks/service.ts:L237-L247`; `apps/mobile/src/features/tasks/use-task-actions.ts:L29-L43`.

What's wrong: Completing a repeating task clears its `rrule` and creates the next copy. Undo just PATCHes `status` back to open, so the original stays open *without* its rule and the new copy also exists.

Evidence (reproduced): after done then Undo, `original {"status":"open","rrule":null}`, plus a second open “Water plants” with `FREQ=DAILY` due the next day.

Why it matters: The snackbar exists because “a stray tap or swipe can tick the wrong task” (code comment). Using it leaves two tasks behind.

Recommendation: When a task goes from done to open within the same occurrence, soft-delete the generated copy (record `nextId` on the completed row) and restore the rule.

QA-016 · Undo after deleting a task brings back the task but not its subtasks

Medium

Category: Tasks / undo Effort: S

Where: `apps/api/src/modules/tasks/service.ts:L252-L262` (remove cascades to children); `modules/tasks/routes.ts:L42-L44` → `lib/crud.ts:L101-L107` (restore doesn't cascade).

Evidence (reproduced): subtasks before delete: 2 → delete → restore → subtasks after Undo: 0.

Why it matters: The spec chose “Undo snackbar instead of Are you sure?”, so Undo has to be lossless.

Recommendation: In `taskService.restore`, also restore children whose `deletedAt` equals the parent's `deletedAt`.

4.5 Security & access

QA-017 · The fingerprint lock is skipped on cold start: killing and reopening the app bypasses it

High

Category: Security / app lock Effort: S

Where: `apps/mobile/src/features/security/app-lock.tsx:L11-L41`.

What's wrong: `locked` is initialised once with `useState(enabled)`. On a cold start, settings come from the persisted cache, which is restored asynchronously by `PersistQueryClientProvider`, so on first render `enabled` is `false` and `locked` stays false. When settings arrive, the effect can only *unlock*. The lock only arms after a background → foreground transition.

```
const enabled = settings?.appLock ?? false;
const [locked, setLocked] = useState(enabled); // false while settings load
useEffect(() => { if (!enabled) setLocked(false); }, [enabled]); // never re-locks
```

Evidence: code reading. Unverified on device. Steps: enable App lock, swipe the app away from Recents, reopen it. Expected: fingerprint prompt. Predicted: Today opens with amounts visible.

Why it matters: This is the one privacy control for a phone handed to someone else.

Recommendation: Store the lock preference on the device (SecureStore/AsyncStorage) and read it before first render, or lock on mount until settings resolve: `useEffect(() => { if (enabled) setLocked(true) }, [enabled])` for the first resolution. Optionally set `FLAG_SECURE` to hide the Recents thumbnail.

QA-018 · Login rate limit and bind address differ from the spec

Low Category: Security / config Effort: S

Where: `apps/api/src/modules/auth/routes.ts:L24` (10 per 15 min; the spec says 5), `:L27` (`x-real-ip ?? "local"`: every caller without the header shares one bucket); `apps/api/src/env.ts:L5` (`HOST` default `0.0.0.0`; spec p. 26 says the process listens on `127.0.0.1`).

Why it matters: If the README step `HOST=127.0.0.1` is missed, the API is reachable on the raw port without TLS, and `X-Real-IP` can be forged to get past the limiter.

Recommendation: Default `HOST` to `127.0.0.1`. Align the limit with the spec, or update the spec.

4.6 AI

QA-019 · AI guardrails are weaker than the spec: the assistant saves without confirmation and the cap is checked once per turn

Medium Category: AI / spec deviation Effort: M

Where: `apps/api/src/modules/ai/prompts.ts:L46` (“Adds and edits happen right away”); spec p. 37 “Deliberately not building: Any AI feature that saves records without my confirmation” and p. 24 “each returns a draft and never saves”; `apps/api/src/modules/ai/routes.ts:L95` and `assistant/agent.ts:L13`, `:L165-L206` (one `requireAi`, then up to 13 “smart” calls); `apps/api/src/ai/usage.ts:L59-L81` (owner never capped; the cap is a server env var, not a Settings value as the spec says).

What's wrong: The chat assistant creates and edits transactions and tasks directly. Undo exists, but a misheard Bangla voice note can change account balances before the user sees anything. The monthly cap is checked before the first model call, so one assistant turn can run 13 paid calls past the cap. For the owner there is no cost ceiling at all.

Why it matters: It conflicts with “Honest numbers” and with the spec's explicit exclusion. The cost exposure is unbounded for the owner.

Recommendation: Make the choice explicit: either update the spec, or have money writes from the assistant come back as drafts the user confirms (deletes already work this way). Re-check the cap inside the loop. Restore an owner-settable cap in Settings, even if it defaults to “none”.

QA-020 · AI calls can outlive Nginx's default 60 s proxy timeout, and bad AI output returns 500 instead of `ai_error`

Medium

Category: AI / failure handling Effort: S

Where: `apps/api/src/ai/openai-client.ts:L20` (`timeout: 45_000`, `maxRetries: 1`, so up to about 90 s+); `deploy/nginx-tick-taka.conf:L10-L18` (no `proxy_read_timeout`, Nginx default 60 s); `apps/mobile/src/lib/http.ts:L7` (client waits 90 s); `apps/api/src/modules/ai/parse.ts:L169, :L190, :L272` (`schema.parse` runs *outside* `callAi`); `middleware/error-handler.ts` maps unknown errors to 500 `internal_error`.

What's wrong: A slow receipt scan or plan-day call that retries once gets a 504 HTML page from Nginx, which the app shows as “Request failed (504)”. The model is still billed and no usage is logged, because `logUsage` only runs on success. Output that fails the Zod schema surfaces as a generic server error rather than the documented fallback code.

Evidence: config and code reading. Unverified against a live Nginx.

Recommendation: Set `proxy_read_timeout 120s;` for `/ai/`, or cut the server budget to under 55 s in total (`maxRetries: 0, 25 s`). Move `schema.parse` inside `callAi` so every AI failure is a 502 `ai_error`.

QA-021 · Assistant writes continue on the server if the stream drops, without the client showing Undo

Low

Category: AI / streaming Effort: M

Unverified

Where: `apps/api/src/modules/ai/routes.ts:L93-L125` (no abort handling around `aiAssistant`); `apps/mobile/src/lib/event-stream.ts:L52-L58` (a read error becomes `ServerUnreachableError`).

What's wrong: When the phone loses the connection mid-turn, nothing visible stops the agent loop from carrying on with writes through `dispatch`. The client never receives the `action` events, so it can't offer Undo. Not reproduced; Hono's behaviour on client abort for `streamSSE` wasn't tested.

Recommendation: Stop the loop on `stream.onAbort`. After a dropped stream, have the client fetch “recent assistant actions” so Undo is still available.

4.7 Client platform & jobs

QA-022 · Reminder scheduling has gaps, and permission denials are silent

Medium

Category: Client / notifications Effort: M

Where: `apps/mobile/src/features/notifications/use-notification-scheduler.ts:L13-L31, :L91-L99`; `apps/mobile/src/lib/notifications.ts:L12-L17`; `apps/mobile/src/lib/receipts.ts:L19-L22`.

What's wrong: (1) Task reminders are planned only for tasks whose `doAt` falls between now – 1 day and now + 8 days, so an inbox task with a `reminderAt` but no `doAt` never rings. (2) Rescheduling runs only after Today refetches, so if the app isn't opened for about 9 days, no reminders exist. (3) A denied notification or camera permission returns early with no message and no link to Settings. (4) Errors in `reschedule` are swallowed (`.catch(() => {})`).

Why it matters: The spec relies on phone-scheduled reminders, with no push. Missed bill and task reminders are invisible failures.

Recommendation: Query tasks by `reminderAt` range rather than `doAt`. Schedule on a background fetch as well as on app open. When permission is denied, show a one-line banner in Settings with “Open settings”.

QA-023 · Nightly jobs have no catch-up when the server is down at :05

Low

Category: Server / scheduled jobs Effort: S

Where: `apps/api/src/jobs/scheduler.ts:L17-L43`.

What's wrong: Each user's backup runs only if the process is alive at `xx:05` during local hour 3, and overdue marking only during hour 0. A restart or deploy at that moment skips the day, and nothing records or retries it. Overdue *status* is computed from dates anyway, so only the `overdueAt` flag and the backup are affected.

Recommendation: On each hourly tick, back up if today's file is missing and the local hour is ≥ 3 . Log the last success in the `settings` table under an internal `_` key.

QA-024 · SMS auto-capture is missing but still listed in the spec checklist and implementation plan

Low

Category: Traceability / docs Effort: S

Where: spec pp. 17–18 and the build checklist p. 36 (“SMS auto-capture and review panel”, “Receipt and SMS scan”); `docs/IMPLEMENTATION_PLAN.md:L33`, `:L52`, `:L79`; `README.md:L291` (“SMS permissions are blocked in the build”). `git ls-files | grep -i sms` returns nothing.

Why it matters: The plan claims 63 features, which makes the release look complete. The owner should know this was dropped (presumably because of Play/Android SMS policy).

Recommendation: Mark SMS as “dropped” in the spec checklist and plan, with the reason.

4.8 Test suite

QA-025 · The offline/outbox layer and the Bangla, replay and rollover edge cases have no tests

Medium

Category: Testing Effort: M

Where: `apps/mobile/test/` has 5 files (format, optimistic-spend, plan-notifications, server-clock, widget-snapshot), and `_raw/coverage-mobile.txt` lists no `lib/query-client.ts`, `outbox.ts`, `auth.ts` or `user-data.ts`. In `packages/shared/test/quick-add.test.ts` (15 tests) there are no Bangla inputs. `apps/api/test/money.test.ts:L257-L285` tests rollover only when the user re-saves budgets each month. There is no test of PATCH-after-create replay, `/pay` replay, timer start/stop with explicit times, or delete → restore with subtasks.

Quality of existing tests: Good. The assertions check real values (for example `dailyMinor: 64_000` for the spec example, exact `nextDueAt` instants, user isolation). Fakes are injected through `Deps` rather than mocked globally, and the clock is controllable. No flaky patterns were seen, and nothing depends on the wall clock except token iat/exp in `test/helpers.ts:L71-L72`, which is harmless.

Why it matters: Every Critical and most High findings in this report sit in code paths no test reaches.

Recommendation: See the test plan in section 7. Extract the outbox retry/persist policy into a pure module so it can be unit-tested with Bun.

QA-026 · Coverage figures for `dates.ts` and `money.ts` are unreliable

Low

Category: Testing / tooling Effort: S

Where: `_raw/coverage-shared.txt`: `src/dates.ts` | 79.31 % funcs | 0.00 % lines | 6-211, even though `dates.test.ts` has 6 tests. `_raw/coverage-api.txt`: `money.ts` 1.18 % lines.

What's wrong: Bun's line mapping for these modules is clearly wrong: functions are reported covered while lines are reported uncovered. Coverage can't currently be used as a gate for the most important helpers.

Recommendation: Re-check with a newer Bun or with `--coverage-reporter=lcov` and an external viewer. Don't set coverage thresholds until the numbers are trustworthy.

5. Strengths (keep these)

- **Money is integers end to end.** `toMinor` rounds half away from zero with an epsilon (`money.ts:L28-L33`), `splitEvenly` conserves totals, and balances and budgets are computed from transactions, never stored.
- **Safe-to-spend matches the spec.** Flexible-left is taken as of the start of today, divided by days left including today, and floored to whole taka (`finance.ts:L20-L24`). The API test reproduces the spec's ৳640 example exactly.
- **Client-generated ULIDs make creates idempotent** (`crud.ts:L71-L80` , debts, goals, balance check, receipts), and the phone stamps edits in server time via the `x-server-time` offset. Both are the right foundations; QA-002 and QA-007 are gaps in applying them, not in the design.
- **Per-user database isolation** is enforced in the middleware (`runAsUser`) and covered by `users.test.ts` , and the auth middleware wraps the whole API.
- **AI failures degrade gracefully in quick-add.** `callAi` turns provider errors into 502 `ai_error` , the cap and off switch have their own codes, and `use-quick-add.ts` silently keeps the on-phone draft.
- **Time-zone handling is careful.** Instants are UTC, local dates are strings, `zonedTimeToUtc` does two-pass offset resolution, jobs run per user time zone, and Bangladesh's Sun–Thu work week is honoured.
- **Clean gates:** 0 lint diagnostics, 0 type errors, 185 passing tests with meaningful assertions and an injectable clock.

6. Action plan

Order	ID	Title	Severity	Effort
1	QA-001	Outbox: retry until reachable, persist in-flight writes	Critical	M
2	QA-002	Offline create+edit: stop last-write-wins from dropping the edit	Critical	S
3	QA-003	Don't wipe the queue on 401 or sign-out; refresh sessions	High	M
4	QA-011	Normalise Bangla digits; send Bangla text to AI	High	S
5	QA-007	Make bill "Paid" idempotent; one-tap guard on Today	High	S
6	QA-017	Lock on cold start	High	S
7	QA-004	Timer start/stop carry explicit times	High	S
8	QA-008	Rollover across inherited months	High	S
9	QA-025	Add the P0 tests from section 7	Medium	M
10	QA-013	Date-keyed Today + midnight refetch + stale banner	Medium	S
11	QA-015	Undo completion of a repeating task	Medium	S
12	QA-016	Restore subtasks with parent	Medium	S
13	QA-012	Tighten recurrence and stray-number heuristics	Medium	S
14	QA-009	Exclude or convert foreign-currency spending in budgets	Medium	M
15	QA-020	AI timeouts vs Nginx; schema errors → ai_error	Medium	S
16	QA-019	Decide on assistant confirmation and cap enforcement	Medium	M
17	QA-022	Reminder scheduling by reminderAt; permission banners	Medium	M
18	QA-005	Idempotent checkout / stop / restore	Low	S
19	QA-014	Validate RRULE ranges and reachability	Low	S
20	QA-018	HOST default 127.0.0.1; align login limit	Low	S
21	QA-023	Backup catch-up	Low	S
22	QA-010	Calculator rounding vs spec	Low	S
23	QA-021	Stop assistant loop on client abort	Low	M
24	QA-006	Use or retire /sync/changes	Low	M
25	QA-024	Mark SMS as dropped in docs	Low	S
26	QA-026	Fix coverage mapping before using thresholds	Low	S

7. Recommended test plan

Priority P0 covers the Critical/High findings, P1 the Medium ones, P2 hardening. “Unit” runs with `bun test` in shared or mobile; “API” uses `createTestContext` with in-memory SQLite and a controllable clock; “E2E” needs a device or emulator (for example Maestro).

Pri	Type	Covers	Given / When / Then
P0	Unit (mobile)	QA-001	Given the outbox retry policy, when the error is <code>ServerUnreachableError</code> or <code>ApiError(502 503 504 429)</code> , then the policy says retry for any attempt count; when the error is <code>ApiError(400 404 409)</code> , then it gives up.
P0	E2E	QA-001	Given Wi-Fi on and the API stopped, when the user logs “cha 20”, waits 60 s, kills the app, restarts the API and reopens the app, then exactly one ₳20 expense exists on the server.
P0	API	QA-002	Given a task created with id X at replay time T, when a PATCH arrives with <code>updatedAt = T - 2 h + 1 min</code> (an edit made offline after the create), then the title is updated.
P0	Unit (mobile)	QA-003	Given two queued writes, when a 401 is received, then the mutation cache still holds both and the user is asked to sign in again.
P0	Unit (shared)	QA-011	Given default context, when parsing “চা ২০”, “lunch ২৫০”, “রিকশা ৬০ bkash”, “২h thesis”, “১৮৫০/৩”, then the results are expense ₳20, expense ₳250, expense ₳60 from bKash, a 120-minute time entry, and ₳616.67 (or ₳617, per QA-010).
P0	API	QA-007	Given a monthly bill due on the 5th, when <code>/pay</code> is called twice with the same <code>transactionId</code> , then one expense exists and <code>nextDueAt</code> moved forward exactly one month.
P0	API	QA-004	Given <code>/timer/start</code> with <code>startedAt = S</code> , when <code>/timer/stop</code> with <code>endedAt = S + 45 min</code> arrives 2 h later, then the entry is 45 min and nothing is running; when stop is replayed, then it returns 200 with the same entry.
P0	API	QA-008	Given September budgets with rollover and no later saves, when reading October, November and December, then each month's <code>carriedMinor</code> equals the previous month's unspent amount.
P0	Component (mobile)	QA-017	Given persisted settings with <code>appLock: true</code> , when <code>AppLock</code> mounts before settings resolve, then the lock screen renders and <code>authenticateAsync</code> is called.
P1	API	QA-015	Given a daily task, when completed and then set back to open, then exactly one open task remains and it keeps <code>FREQ=DAILY</code> .
P1	API	QA-016	Given a task with 2 subtasks, when deleted and restored, then both subtasks are live again.
P1	Unit (shared)	QA-012	Given “submit weekly report”, when parsed, then <code>rrule</code> is null and the title keeps “weekly”; given “every week call mom”, then <code>FREQ=WEEKLY</code> .
P1	API	QA-009	Given a Food budget and a USD expense, when reading safe-to-spend, then the USD amount is either excluded or converted, never added raw.

Pri	Type	Covers	Given / When / Then
P1	Unit (mobile)	QA-013	Given cached Today data for 2026-10-05 and a clock at 2026-10-06 00:30 Dhaka, when the Today hook runs offline, then a stale banner is shown and a new expense updates a today-dated view.
P1	API	QA-020	Given the fake AI returns JSON that fails <code>aiParseOutputSchema</code> , when calling <code>/ai/parse</code> , then the response is 502 <code>ai_error</code> , not 500.
P1	Unit (mobile)	QA-022	Given an inbox task with <code>reminderAt</code> tomorrow and <code>doAt</code> null, when <code>planNotifications</code> runs, then a reminder is planned.
P2	Unit (shared)	QA-014	Given <code>FREQ=MONTHLY;BYMONTHDAY=0</code> , when validated with <code>rruleSchema</code> , then it is rejected.
P2	Unit (shared)	dates	Given <code>Asia/Kathmandu (+5:45)</code> and <code>America/New_York</code> across a DST change, when computing <code>localDayRange</code> and <code>daysLeftInMonth</code> on the 1st and last day, then ranges are contiguous and 23/25-hour days are handled.
P2	API	QA-005	Given a checkout already done with <code>transactionId X</code> , when it is replayed, then 200 with the same transaction.

8. Manual smoke test (≈15 minutes, real Android phone)

Use a fresh day, the production APK and the VPS API. Tick each line. Steps marked $\triangle$ target findings in this report, so a failure there confirms the finding.

#	Step	Expected
1	Cold-start the app while signed in.	Today loads within about 2 s from cache. Date in the header is today.
2	Quick-add “cha 20”, then Save.	Preview “Expense ₳20 · Food · Cash”. Safe-to-spend drops by ₳20 at once.
3 $\triangle$	Quick-add “চা ২০” with the Bangla keyboard.	An expense of ₳20, not a task (QA-011).
4	Quick-add “+45000 salary” and “2h thesis”.	Income ₳45,000; a 2 h time entry under Research.
5 $\triangle$	Quick-add “submit weekly report”.	A one-off task titled “Submit weekly report” (QA-012).
6	Turn on airplane mode. Log “rickshaw 60 bKash”, then open it and change the amount to 70.	The pending-writes indicator shows 2.
7 $\triangle$	Turn airplane mode off and pull to refresh on Money › Transactions.	One rickshaw expense of ₳70 from bKash (QA-002).
8 $\triangle$	Stay on Wi-Fi, have someone run <code>pm2 stop tick-taka-api</code> , log “tea 15”, wait 20 s, then <code>pm2 start</code> .	The expense syncs, with no “Couldn't save” toast (QA-001).
9	Tick a top-three task, then tap Undo.	Task back to open; no duplicate if it repeats (QA-015).
10 $\triangle$	Delete a task with subtasks, then Undo.	Task and subtasks are back (QA-016).
11 $\triangle$	Mark a due bill “Paid” on Today, tap twice quickly.	One expense; the bill moves to next month, not the month after (QA-007).
12	Start a timer under Time, stop it after 1 min.	A 1 min entry; the running bar disappears.
13 $\triangle$	Settings › enable App lock. Swipe the app away from Recents and reopen.	Fingerprint prompt before any content (QA-017).
14	Turn on privacy mode on Today.	All amounts hidden.
15	Open the assistant and say “আজ কী কী কাজ আছে?”.	A reply in Bangla listing today's tasks; no records changed.
16	Settings › Export.	A JSON file is saved; it opens and contains <code>transactions</code> .

Appendix A — Raw command output (trimmed)

Lint (bunx biome check .)

```
Checked 388 files in 367ms. No fixes applied.
exit 0
```

Typecheck (bun run typecheck)

```
@tick-taka/shared typecheck: Exited with code 0
@tick-taka/api typecheck: Exited with code 0
@tick-taka/mobile typecheck: Exited with code 0
```

Tests (bun run test)

```
@tick-taka/shared test: 71 pass 0 fail 173 expect() calls (10 files)
@tick-taka/mobile test: 14 pass 0 fail 23 expect() calls (5 files)
@tick-taka/api test: 100 pass 0 fail 332 expect() calls (9 files)
(the "[ai] call failed ... provider down" stack trace is an expected log from ai.test.ts)
```

Coverage, selected lines

```
shared All files 76.54 % funcs | 88.82 % lines
  src/dates.ts 79.31 | 0.00 | 6-211 <- see QA-026
  src/quick-add.ts 91.18 | 84.48 | 107,109-110,250-251,293-294,362,417-426,440-473
  src/recurrence.ts 97.06 | 96.86
mobile All files 81.73 % funcs | 71.63 % lines (only 5 pure modules measured)
api All files 86.29 % funcs | 93.60 % lines
  src/modules/ai/ask.ts 48.15 | 62.68
  src/modules/ai/assistant/tools.ts 93.75 | 73.46 | 343-415,423-454
  src/middleware/error-handler.ts 100.00 | 72.00 | 9,17,22-26
  ../../packages/shared/src/recurrence.ts 53.33 | 72.21
```

Dependency audit (bun audit)

```
braces <=3.0.3 high via tailwindcss, expo, react-native, react-native-worklets
node-forge <=1.4.0 high via expo
postcss-selector-parser, decode-uri-component, esbuild (drizzle-kit, tailwindcss), uuid
moderate
6 vulnerabilities (2 high, 4 moderate) – exit 1
```

Appendix B — QA reproduction output

Scripts imported the project's own modules and `apps/api/test/helpers.ts` (in-memory SQLite, injected clock at 2026-10-04 10:00 Dhaka). They lived in `docs/audit/2026-10-06/_qa_tmp/`, were run with `bun` from `apps/api`, and were deleted afterwards. No source file was changed.

```

== shared parser (packages/shared) ==
parseQuickAdd("cha 20")           {"kind":"expense","amountMinor":2000,"confidence":"high"}
parseQuickAdd("চা ২০")           {"kind":"task","title":"চা
২০","rrule":null,"confidence":"high"}
parseQuickAdd("lunch ২৫০")       {"kind":"task","title":"Lunch
২৫০","rrule":null,"confidence":"high"}
parseQuickAdd("রিকশা ৬০ bkash") {"kind":"task","title":"রিকশা ৬০
bkash","rrule":null,"confidence":"high"}
parseQuickAdd("২h thesis")       {"kind":"task","title":"২h
thesis","rrule":null,"confidence":"high"}
parseQuickAdd("submit weekly report") {"kind":"task","title":"Submit
report","rrule":"FREQ=WEEKLY","confidence":"high"}
parseQuickAdd("pay monthly rent") {"kind":"task","title":"Pay
rent","rrule":"FREQ=MONTHLY","confidence":"high"}
parseQuickAdd("read chapter 5")   {"kind":"expense","amountMinor":500,"confidence":"low"}
parseAmountToMinor("২৫০")        null
parseRecurrence("every month on the 0th") "FREQ=MONTHLY;BYMONTHDAY=0"
nextOccurrence(BYMONTHDAY=0)      null

```

```

== API (apps/api, in-memory) ==
1 PATCH status: 200
1 title after replay: "Buy milk" (QA-002)
2 first pay nextDue: "2026-11-05T03:00:00.000Z"
2 replay nextDue: "2026-12-05T03:00:00.000Z"
2 expense rows: 1 (QA-007)
3 2026-10 inherited=true: {"limit":300000,"carried":300000,"rollover":true}
3 2026-11 inherited=true: {"limit":300000,"carried":0,"rollover":true}
3 2026-12 inherited=true: {"limit":300000,"carried":0,"rollover":true} (QA-008)
4 spentToday after $50 USD expense (minor, read as BDT): 5000
4 leftToday before/after: [42800,37800] (QA-009)
5 original after undo: {"status":"open","rrule":null}
5 open 'Water plants' tasks: [{"rrule":null,"doAt":"2026-10-03T18:00:00.000Z"},
{"rrule":"FREQ=DAILY","doAt":"2026-10-04T18:00:00.000Z"}]
(QA-015)
6 unauthenticated unknown path: 401
6 unauthenticated /budgets/safe-to-spend: 401 (auth check)
timer: start 201 startedAt - tapStart (min): 120
timer: stop 400 {"error":{"code":"bad_request","message":"Stop time must be after the
start"}}
timer: still running on server: true (QA-004)
restore: subtasks before delete: 2
restore: restored task status: 200 subtasks after Undo: 0 (QA-016)

```

Appendix C — Requirement checklist used

From PRODUCT.md and the spec: log anything in under 5 s; Today answers “what now?” and “can I afford it?”; safe to spend = flexible left ÷ days left including today, fixed bills excluded; amounts in integer minor units; offline cache with queued writes sent in order, IDs from the phone, last-write-wins on `updated_at` ; Asia/Dhaka for display, local strings for habit dates and budget months; streaks with 2 freezes a month; reward, never guilt; honest numbers (balances and hours reconcile); AI: rules first, cost cap, graceful fallback, minimum data, off switch, never saves without confirmation; app lock after 5 min; receipts served only to authenticated requests; login limited to 5 / 15 min; Bun listens on 127.0.0.1; nightly VACUUM INTO backup keeping 14; JSON export; local notifications with quiet hours 23:00–07:00; Bangla and English quick-add. Each item is mapped in section 3.

Appendix D — File inventory reviewed

Area	Files
Shared	packages/shared/src/{money, dates, finance, calculator, quick-add, recurrence, gamification, ids}.ts, schemas/{common, money}.ts
API core	apps/api/src/{app, index, env}.ts, middleware/{auth, rate-limit, error-handler}.ts, lib/{crud, resource-routes, validate, user-time, money-queries, deps}.ts, jobs/{scheduler, backup}.ts, db/user-registry.ts
API modules	auth, budgets, today, transactions, accounts, recurring, tasks, time-entries, goals, debts, shopping, sync, uploads, habits (service), insights (summary, grep)
API AI	ai/{usage, openai-client}.ts, modules/ai/{routes, parse, prompts}.ts, assistant/agent.ts (loop limits)
Mobile	src/lib/{query-client, outbox, http, api, queries, auth, user-data, connection, server-clock, optimistic-spend, today-cache, use-remove, notifications, receipts, event-stream}.ts; app/_layout.tsx, app/(tabs)/index.tsx; features/quick-add/*, today/{next-up, safe-to-spend-card}.tsx, tasks/use-task-actions.ts, money/recurring-sheet.tsx, time/time-tracking.tsx, focus/focus-screen.tsx, security/app-lock.tsx, notifications/use-notification-scheduler.ts, settings/account-section.tsx, components/ui/amount-keypad.tsx
Tests	apps/api/test/{helpers, money-helpers, money}.ts; packages/shared/test/quick-add.test.ts; test-file inventory of all workspaces
Deploy / docs	deploy/nginx-tick-taka.conf, README.md, PRODUCT.md, DESIGN.md, docs/IMPLEMENTATION_PLAN.md, spec (via _raw/spec.txt)